

深圳大学实验报告

课程名称： 软件体系结构与设计模式

实验项目名称： 实验 4 行为型设计模式分析与应用

学院： 计算机与软件学院

专业： 软件工程

指导教师： 毛斐巧

报告人： 学号： 班级：

实验时间： 2026 年 6 月 3 日-2026 年 6 月 30 日

实验报告提交时间： 2026 年 6 月 29 日

教务部制

一、 实验目的与要求：

- 1. 理论联系实际理解 GoF 23 个设计模式中的行为型设计模式。
- 2. 理解每一种行为型设计模式的模式动机，掌握行为型设计模式的灵魂，学习在何种问题背景下，编写代码来恰当地实现这些设计模式。
- 3. 学会综合应用创建型、结构型和行为型设计模式解决现实的软件设计问题。

二、 实验内容

回顾作业 1、实验 2 和实验 3 的内容：

作业 1 主要实现以物料品号为关键字的光缆工艺参数向量<缆芯外径、护套外径、挤出内模、挤出外模、螺杆速度、螺杆电流、生产速度和实际生产速度>的数据提取与初步统计；

实验 2 对《HT_工序纪录明细查询表》，新增需要记录的参数：物料品号、创建日期、设备名称、备注信息。新增品质检验表的导入需求；

实验 3 对《品质检验数据表》，新增要求提取定性字段的值、定量字段的上下界值，判断 isOK 是否与数据统计结果一致。

注：请务必在报告中展示和简述试运行效果和自己的编码逻辑。

1. 减少的功能需求

本次实验要求去除“备注信息”处理需求，包括统计每个操手机的纯度信息。

2. 新增功能需求

2.1) 每析定量字段中哪些字段的值与<缆芯外径、护套外径、挤出内模、挤出外模、螺杆速度、螺杆电流、生产速度和实际生产速度>之中的哪一个相关？，要求给出如下形式的 8x10 关联分析表：

	定量字段 1， 如《套管外径》	定量字段 2	..	...	...	定量字段 10
缆芯外径	正相关或负 相关度量	正相关或负 相关度量				
护套外径						

其中正相关或负相关的定义，请查阅 Pearson correlation coefficient https://en.wikipedia.org/wiki/Pearson_correlation_coefficient 或询问大模型，找到一个合适的度量指标。注意：统计样本是在所有可用的《HT_工序纪录明细查询表》和《品质检验数据表》中的数据之上建立的，因此批次越多越好。可以在考虑物料品号和不考虑物料品号的基础上，做上述分析。为此，新增若干《HT_工序纪录明细查询表》和《品质检验数据表》数据表，以提供充分的数据用于统计。

2.2) 对 2.1)表中最有代表性的 5 个定量字段，分别建立一个回归预测器，用<缆芯外径、

护套外径、挤出内模、挤出外模、螺杆速度、螺杆电流、生产速度和实际生产速度>作为输入（可能还包括物料品号、光缆型号），回归预测这 5 个定量字段的值。要求给出预测的**准确性度量**，如 **precision** 和 **recall** 值。具体算法实现，可基于大模型实现。提示：简单模型可采用 Random Forest，并使用 scikit-learn 库实现。

三、理解和分析报告、实践过程及结果等

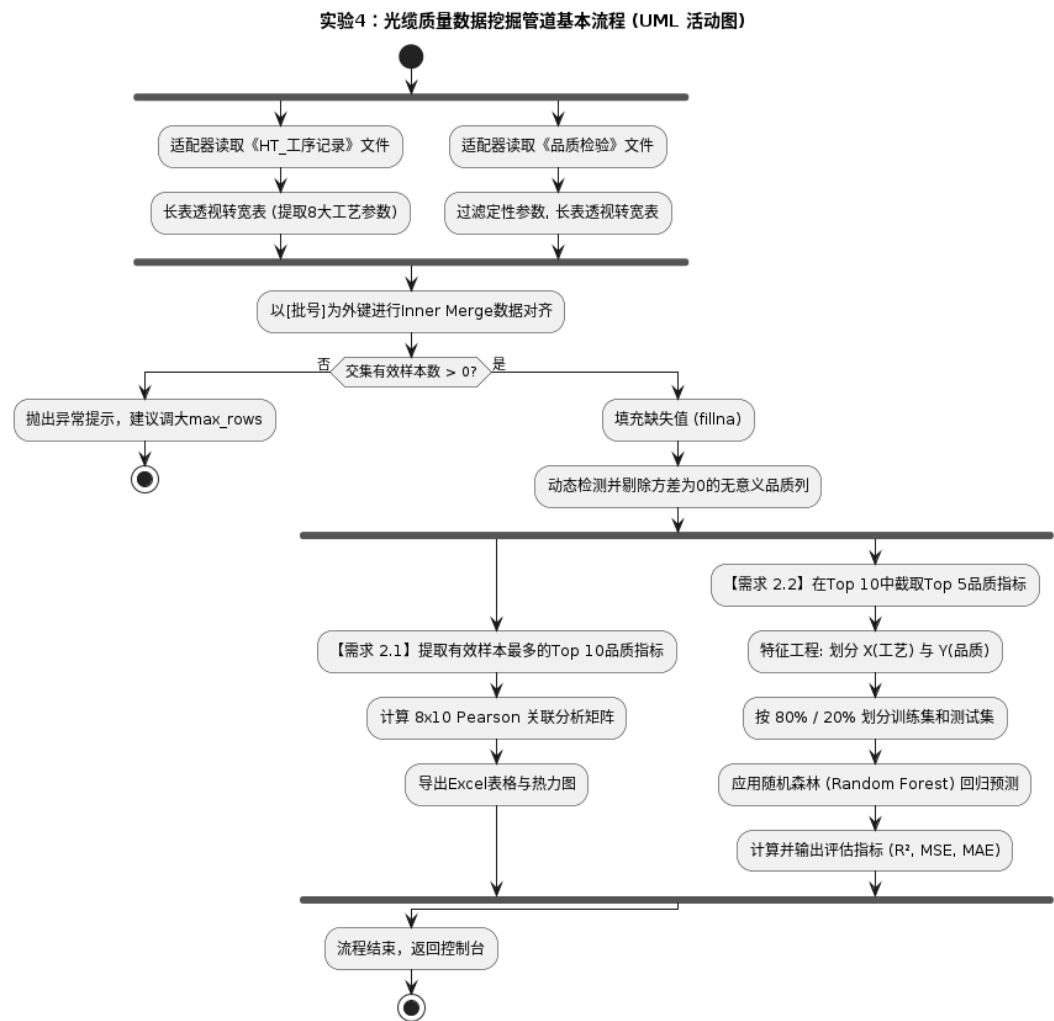
1) 在融合作业 1、实验 2、实验 3 和本次实验对应的 4 个代码库，并重构整体代码库，使代码库可以支持作业 1、实验 2、实验 3 和本次实验的所有需求。

要求给出 UML 类图作为佐证，并填写下述类的功能简述表：

类	实验 1	实验 2	实验 3	实验 4
DataLoader/Adapter 类	面向过程读取：在全局脚本中写死行、列索引，直接读取 Excel 数据，无封装。	OOP 数据获取层：封装为简单的数据读取类，提供基础的数据行迭代功能。	适配器模式：引入 ExcelAdapter，统一处理《工序表》和《品质表》的异构格式读取。	EAV 数据透视：升级为 RealDataExcelAdapter，增加长表转宽表（Pivot）、定性数据剔除功能。
Validation/Rules 类	硬编码校验：在读取循环中充斥着海量的 if/else 空值与类型判断。	策略模式校验：抽象出 IValidationRule，将非空、逻辑比对封装为独立的规则类。	扩展复合校验：支持对新增的品质检验记录进行上下界检验并验证 isOK 字段。	方差与交集保护：继承并延伸了数据清洗能力，新增对方差为 0 的列进行自动剔除，防止计算报错。
Statistics/Pipeline 类	简单字典统计：依靠嵌套 dict 和 for 循环统计各个参数的出现次数。	泛型纯度提取：将统计逻辑抽象，支持传入“物料品号”或“操作机手”作为维度进行纯度计算。	组合与装饰器：使用组合模式管理定量/定性参数树，使用装饰器模式动态扩展综合评价等指标。	模板方法模式：彻底剥离“操作手”等逻辑，升级为 DataAnalysisPipeline，死死锁定加载、清洗、关联、建模这 4 步生命周期骨架。
ML Strategy 类 (本次实验新增)	无	无	无	策略模式建模：新增 IRegressionStrategy 接口及其实现类 RandomForestStrategy，无缝接入 Scikit-learn 模型进行回归预测。

要求使用 UML 活动图绘制整体代码库的基本流程，包括起始结点、中间结点、结点之

间的判定条件、结点之间的并发条件（如果有）、终止结点。



1. 起始与并发数据加载阶段（Fork 节点）

流程从顶部的起始结点出发，首先遇到一个并发分叉（Fork），系统在此处兵分两路，并行处理两类异构数据源：

路径 A：调用适配器读取《HT_工序记录》文件，将原始的长表数据透视转换为宽表，并精准提取出缆芯外径、护套外径等 8 大工艺参数。

路径 B：调用适配器读取《品质检验》文件，首先过滤掉无数学意义的定性参数（如“OK”），然后将剩余的定量数据同样进行长表转宽表的透视操作。

两个并发任务完成后，在下方的合并结点（Join）汇合，等待统一处理。

2. 数据对齐与条件判定阶段（Decision 节点）

汇合后，系统以“批号”作为外键，对两张宽表执行 Inner Merge 操作以对齐数据。随后，流程进入一个条件判定结点（菱形逻辑），检查“交集有效样本数 > 0”是否成立：

判定为“否”：说明当前截取的数据中两表没有批号交集，系统走向异常处理分支，抛出提示“建议调大 max_rows”，并直接到达终止结点，提前结束本次流程。

判定为“是”：说明合并成功，流程进入主干道，继续后续的数据挖掘任务。

3. 数据清洗与特征筛选阶段

主干流程继续向下，首先对合并后的数据执行填充缺失值（`fillna`）操作，以保护小样本交集；紧接着执行关键的清洗逻辑——动态检测并剔除方差为 0 的无意义品质列（如检验值全部为 2.20 的常数列），从而消灭后续皮尔逊计算中可能出现的 NaN 异常。

4. 并发业务需求执行阶段（Fork 节点）

数据清洗完毕后，流程第二次进入并发分叉，同时向下推进本次实验的两个核心需求：

左侧分支（实现需求 2.1）：系统自动提取有效样本最多的 Top 10 品质指标，与 8 大工艺参数计算得出 8×10 Pearson 关联分析矩阵，最后将结果自动导出为 Excel 表格与热力图图片。

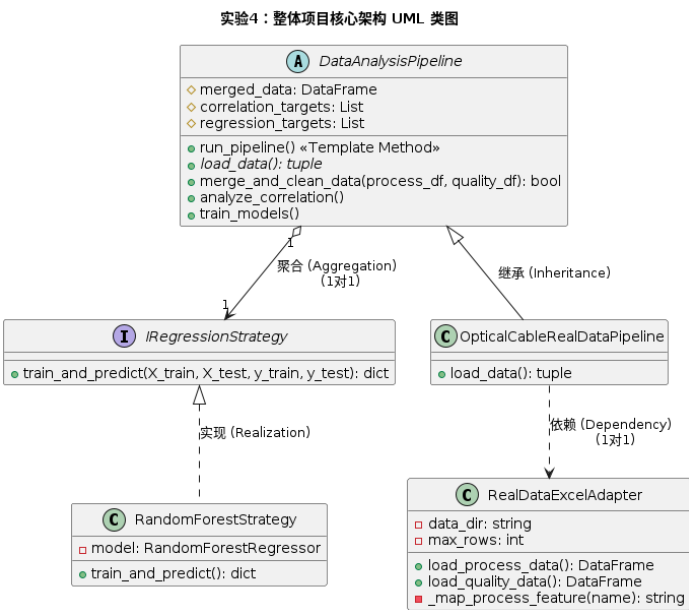
右侧分支（实现需求 2.2）：系统在 Top 10 的基础上进一步截取最具代表性的 Top 5 品质指标作为预测目标。随后进入机器学习环节，进行特征工程划分 X（工艺特征）与

Y（品质目标），按 80% / 20% 比例切分训练集与测试集，应用随机森林（Random Forest）回归算法进行模型训练与预测，最后计算并向控制台输出决定系数 R^2 、均方误差 MSE 和平均绝对误差 MAE 等核心评估指标。

5. 汇合与结束

当需求 2.1 和 2.2 的两个分支均执行完毕后，控制流在底部的合并结点（Join）汇合，系统输出“流程结束，返回控制台”，并最终到达底部的终止结点，标志着整个数据挖掘管道生命周期的圆满完成。

2）使用 UML 的类图绘制整体项目所有类之间的结构关系，要求给出重数、是否采用继承，还是采用组合，1 对 N、N 对 N 或 N 对 1 的关系，哪些类之间存在依赖关系，以及采用这些 UML 关系（继承、关联、依赖）的依据是什么。



继承/实现：OpticalCableRealDataPipeline 继承自 DataAnalysisPipeline，这是模板方法模式的体现。父类定义了整个清洗到机器学习的流水线骨架，子类只重写 load_data 实现真实数据的具体加载。RandomForestStrategy 实现 IRegressionStrategy 接口，满足里氏替换原则，便于未来扩展 XGBoost 等其他算法。

聚合（1 对 1）：DataAnalysisPipeline 中包含了一个 IRegressionStrategy 的实例引用（通过构造函数依赖注入）。流水线“拥有”一个算法策略，但算法策略可以独立于流水线存在，因此是聚合关系，这是策略模式的核心。

依赖：OpticalCableRealDataPipeline 在 load_data 方法中临时实例化并调用了 RealDataExcelAdapter 来获取数据。适配器并不作为类的全局成员变量长期存在，而是作为工具被短暂使用，因此它们之间是弱耦合的依赖关系。

全局依赖与常量定义：

导入数据分析、绘图、机器学习库；配置中文绘图环境；定义 8 个固定工艺特征常量，统一全局特征名，避免硬编码字符串。

```
# =====
# 全局配置
# =====
plt.rcParams["font.sans-serif"] = ["SimHei", "Arial Unicode MS"] # 支持中文显示
plt.rcParams["axes.unicode_minus"] = False

# 标准化的8个工艺参数名称
PROCESS_FEATURES = [
    '缆芯外径', '护套外径', '挤出内模', '挤出外模',
    '螺杆速度', '螺杆电流', '生产速度', '实际生产速度'
]
```

适配器模式 RealDataExcelAdapter（数据读取、长表转宽表）：

构造函数传入数据目录、最大读取行数，支持批量读取多份 Excel 扩充样本；

_map_process_feature 做工序项目名称标准化映射，过滤无关参数；

load_process_data：读取工序长表，按批号透视，聚合均值生成工艺宽表；

load_quality_data：读取品检长表，统一添加[检验]前缀区分字段，过滤非数值定性数据，透视生成检验指标宽表；

统一封装异构 Excel 读取逻辑，符合适配器模式，解耦文件 IO 与上层业务流水线。

```

# =====
# 1. 结构型模式：数据读取适配器
# =====
class RealDataExcelAdapter:
    解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
    def __init__(self, data_dir='/workspace/', max_rows=1000):
        self.data_dir = data_dir
        self.max_rows = max_rows

    解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
    def _map_process_feature(self, name):
        name = str(name).strip()
        if '内护套' in name: return None
        if '缆芯外径' in name: return '缆芯外径'
        if '护套外径' in name: return '护套外径'
        if '挤出内模' in name: return '挤出内模'
        if '挤出外模' in name: return '挤出外模'
        if '螺杆速度' in name and '挤塑' in name: return '螺杆速度'
        if '螺杆电流' in name: return '螺杆电流'
        if '实际生产速度' in name: return '实际生产速度'
        if '生产速度' in name: return '生产速度'
        return None

```

策略模式 IRegressionStrategy + RandomForestStrategy（机器学习回归）：

编码逻辑（实验 4 新增 ML 策略模块，行为型 - 策略模式）：

抽象接口 IRegressionStrategy 统一定义训练预测方法，满足开闭原则；

RandomForestStrategy 实现接口，封装随机森林训练、预测、指标计算；

后续扩展 XGBoost、线性回归仅新增实现类，流水线无需修改，符合里氏替换；

输出 R^2 、MSE、MAE 三类回归评估指标，匹配实验 2.2 预测精度度量要求。

```

# =====
# 2. 行为型模式：策略模式
# =====
class IRegressionStrategy(ABC):
    @abstractmethod
    解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
    def train_and_predict(self, X_train, X_test, y_train, y_test) -> dict:
        pass

class RandomForestStrategy(IRegressionStrategy):
    解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
    def __init__(self, n_estimators=100, random_state=42):
        self.model = RandomForestRegressor(n_estimators=n_estimators, random_state=random_state, n_jobs=-1)

    解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
    def train_and_predict(self, X_train, X_test, y_train, y_test) -> dict:
        self.model.fit(X_train, y_train)
        y_pred = self.model.predict(X_test)

        return {
            "R2_Score": r2_score(y_test, y_pred),
            "MSE": mean_squared_error(y_test, y_pred),
            "MAE": mean_absolute_error(y_test, y_pred)
        }

```


模板方法模式 `DataAnalysisPipeline` 流水线父类（核心业务骨架）：

构造函数注入回归策略对象（聚合关系），实现算法与业务解耦：

`run_pipeline` 固定执行顺序：加载数据→合并清洗→相关性分析→模型训练；

抽象 `load_data` 由子类实现，适配真实 Excel 数据源；

`merge_and_clean_data`：批号内连接合并、缺失值填充、剔除方差为 0 常量列、筛选 Top10/Top5 检验指标；

`analyze_correlation`：计算 Pearson 相关系数，截取 8×10 矩阵，打印、导出 Excel、绘制热力图（需求 2.1 完整实现）；

`train_models`：循环 Top5 指标划分训练测试集，调用策略类训练回归模型，输出评估指标（需求 2.2 完整实现）。

```
# =====
# 3. 行为型模式：模板方法模式（流水线 Pipeline）
# =====
class DataAnalysisPipeline(ABC):
    解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
    def __init__(self, ml_strategy: IRegressionStrategy):
        self.ml_strategy = ml_strategy
        self.merged_data = None
        self.correlation_targets = [] # 需求2.1: 存储 10 个质量指标
        self.regression_targets = [] # 需求2.2: 存储 5 个质量指标

    解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
    def run_pipeline(self):
        print("="*80)
        print("🚀 启动光缆质量数据挖掘管道... (严格匹配实验报告要求)")
        print("="*80)

        process_df, quality_df = self.load_data()
        success = self.merge_and_clean_data(process_df, quality_df)

        if success:
            self.analyze_correlation()
            self.train_models()

        print("\n✅ 所有流水线任务执行完毕。")

    @abstractmethod
    解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
    def load_data(self) -> tuple:
        pass
```


流水线子类 OpticalCableRealDataPipeline（实现抽象加载方法）：

继承流水线父类，重写 load_data，实例化适配器读取真实 Excel，完成模板方法的具体数据接入；可修改 max_rows 调整读取样本量，保证充足数据用于相关性与建模。

```
# =====
# 4. 具体的流水线实现：
# =====
class OpticalCableRealDataPipeline(DataAnalysisPipeline):
    解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
    def load_data(self) -> tuple:
        print("\n[1/4] 开始扫描并加载 /workspace/ 目录下的 Excel 文件...")
        # 建议保持 5000 行，以确保在剔除方差为 0 的列后，依然能凑齐 10 个有效质量指标
        adapter = RealDataExcelAdapter(data_dir="/workspace/", max_rows=5000)
        process_df = adapter.load_process_data()
        quality_df = adapter.load_quality_data()
        return process_df, quality_df
```

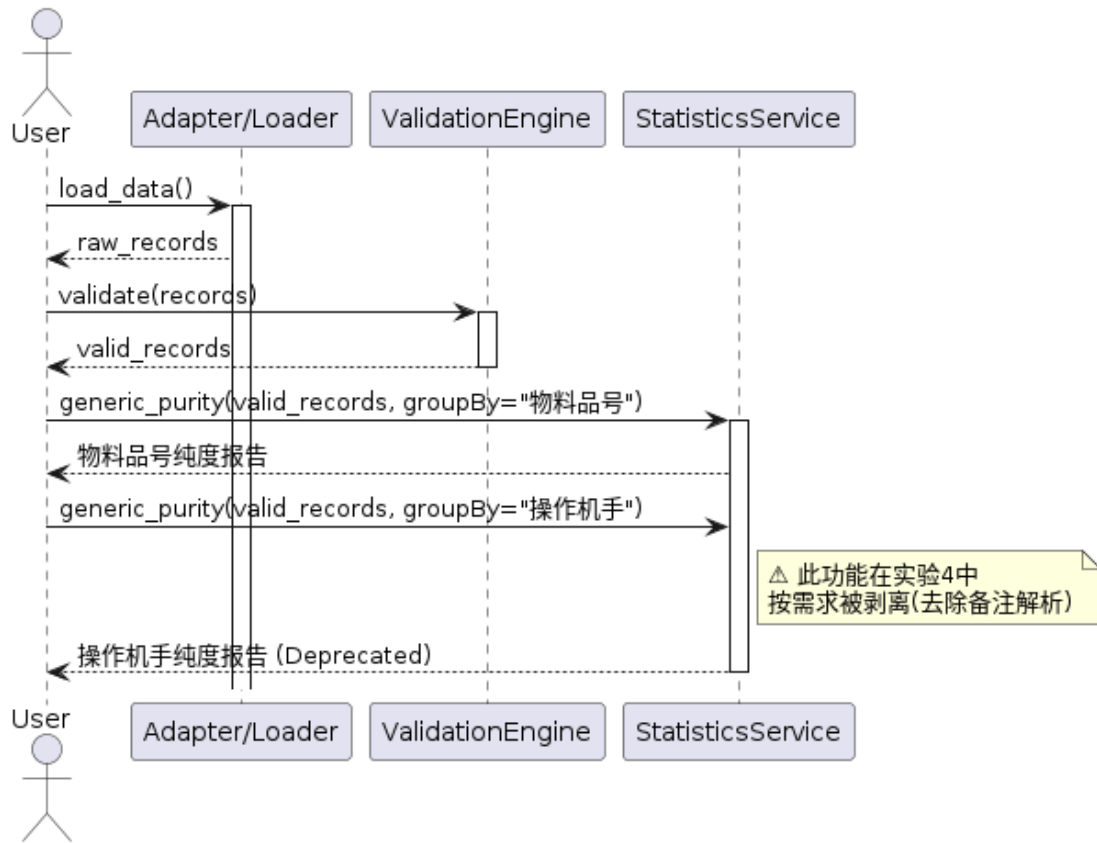
程序入口主函数：

实例化随机森林策略，注入流水线并启动全流程，统一异常捕获打印报错信息，方便调试。

```
# =====
# 5. 主程序启动
# =====
if __name__ == "__main__":
    try:
        rf_strategy = RandomForestStrategy(n_estimators=100)
        pipeline = OpticalCableRealDataPipeline(ml_strategy=rf_strategy)
        pipeline.run_pipeline()
    except Exception as e:
        print(f"\n✗ 程序执行发生异常：{e}")
```

3) 使用 UML 的时序图绘制整体项目所有类之间的通信和协作关系，要求按每个功能分别绘制时序图，已知功能有：每表字段提取、每表字段入库、字段有效性验证、物料品号纯度统计、操作机手纯度统计（尽管本次实验已减去）、8x10 统计相关性矩阵计算、每定量字段回归预测等。

时序图A：数据读取、验证与纯度统计（含实验4剔除逻辑）



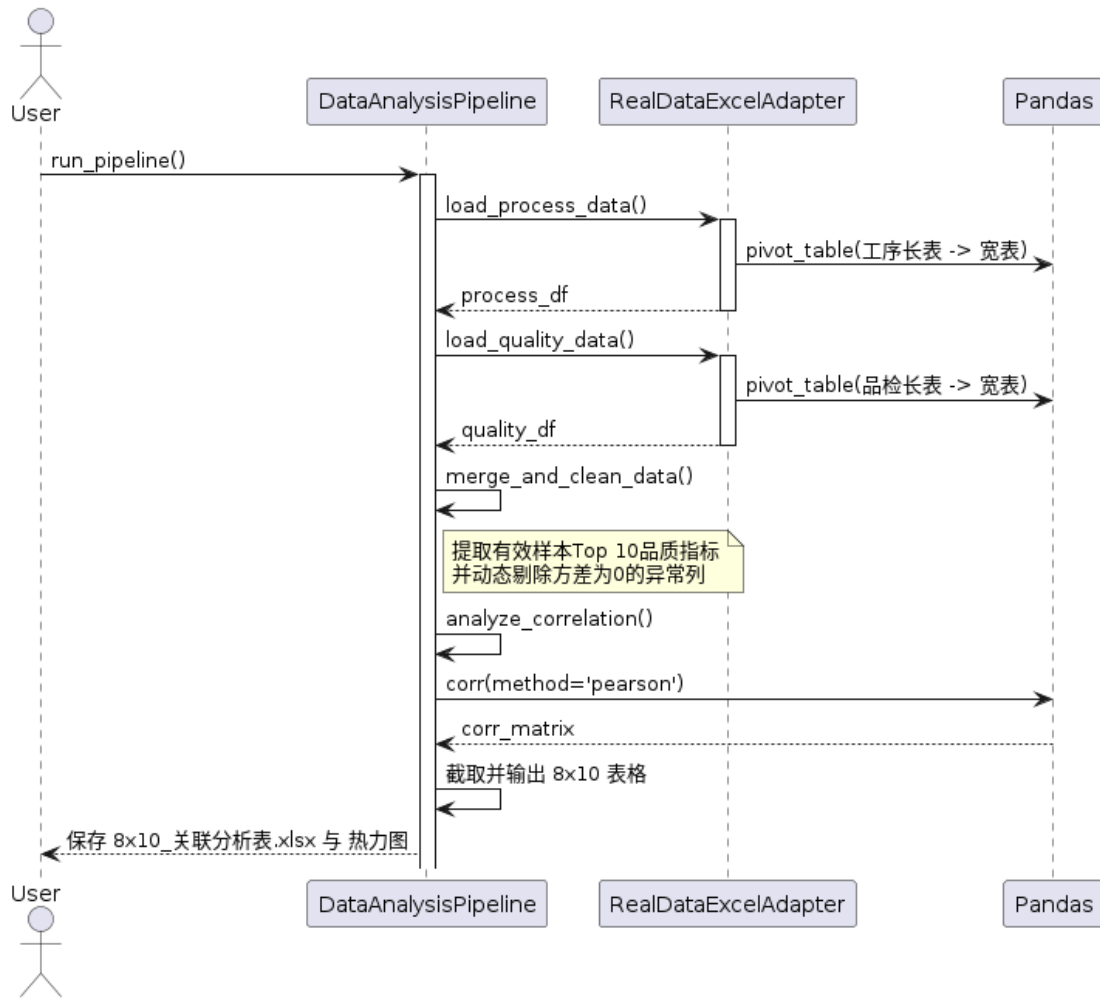
数据获取：用户（User）向 Adapter/Loader（适配器/加载器）发起 load_data() 请求，适配器完成底层文件 I/O 操作后，返回未清洗的原始数据记录（raw_records）。

数据清洗：用户将原始数据交由 ValidationEngine（规则校验引擎）处理，引擎内部调用各个校验规则，最终返回合法的数据记录（valid_records）。

物料品号纯度统计：用户调用 StatisticsService（统计服务）的泛型方法 generic_purity，传入合法的记录并指定 groupBy="物料品号"，服务完成计算并返回物料品号的纯度报告。

功能裁剪注记（操作手纯度统计）：当系统继续请求按“操作机手”进行纯度统计时，时序图中使用了一个备忘录（Note）明确标示：“此功能在实验 4 中按需求被剥离（去除备注解析）”。因此，该调用链路已被标记为 Deprecated（废弃）。这一设计清晰地追踪了系统在需求变更下的功能衰退与演进。

时序图B：8×10 统计相关性矩阵计算（需求2.1）



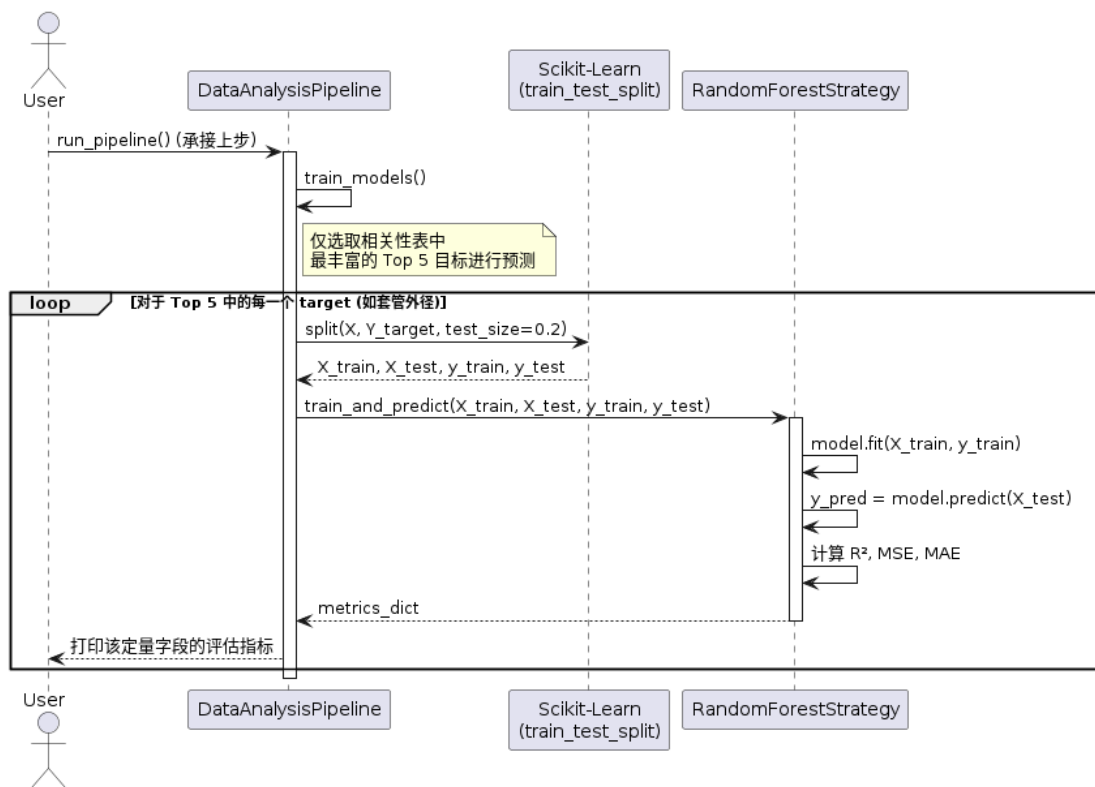
启动管道：用户调用 `DataAnalysisPipeline` 的 `run_pipeline()` 方法，触发整个模板方法的执行。

适配器透视数据：Pipeline 分别请求 `RealDataExcelAdapter` 加载工序数据和品质数据。此时，Adapter 内部调用底层的 `Pandas` 库执行 `pivot_table` 函数，将真实的“长表（一行一个参数）”透视为便于机器学习的“特征宽表（一列一个参数）”，并返回给 Pipeline。

数据对齐与高级清洗：Pipeline 调用自身的 `merge_and_clean_data()` 方法，对两张宽表进行 Inner Merge，并在此时动态提取有效样本量最多的 Top 10 品质指标，同时剔除方差为 0（全为相同常数）的异常列。

相关性计算与输出：Pipeline 调用 `analyze_correlation()` 方法，向 `Pandas` 发起 `corr(method='pearson')` 计算请求。`Pandas` 返回完整的相关系数矩阵后，Pipeline 精准截取出 8×10 的目标表格，最后将 Excel 表格（8x10_关联分析表.xlsx）与热力图保存并返回给用户。

时序图C：Top 5定量字段机器学习回归预测（需求2.2）



承接流水线：Pipeline 在完成相关性分析后，顺次进入 `train_models()` 阶段。系统在此处明确约束：仅选取相关性表中最丰富的 Top 5 目标进行预测建模。

循环建模 (Loop 结构)：针对这 Top 5 的每一个定量字段（如“套管外径”），系统开启一个循环流程（Loop 框）。

特征切分：Pipeline 调用外部库 Scikit-Learn 的 `train_test_split` 方法，将特征 X（工艺参数）和目标 Y 按 80% / 20% 的比例切分为训练集和测试集（`X_train`, `X_test`, `y_train`, `y_test`）。

策略模式执行预测：Pipeline 将切分好的数据集传递给事先注入的策略类 `RandomForestStrategy`。

算法内部闭环：`RandomForestStrategy` 内部依次执行 `model.fit()` 进行模型训练，执行 `model.predict()` 得出预测值，随后计算出回归任务专属的评估指标（决定系数 R^2 、均方误差 MSE 和平均绝对误差 MAE）。

结果返回：策略类将打包好的 `metrics_dict` 返回给 Pipeline，Pipeline 最终将该定量字段的评估指标打印输出给用户，当前循环结束，继续预测下一个目标。

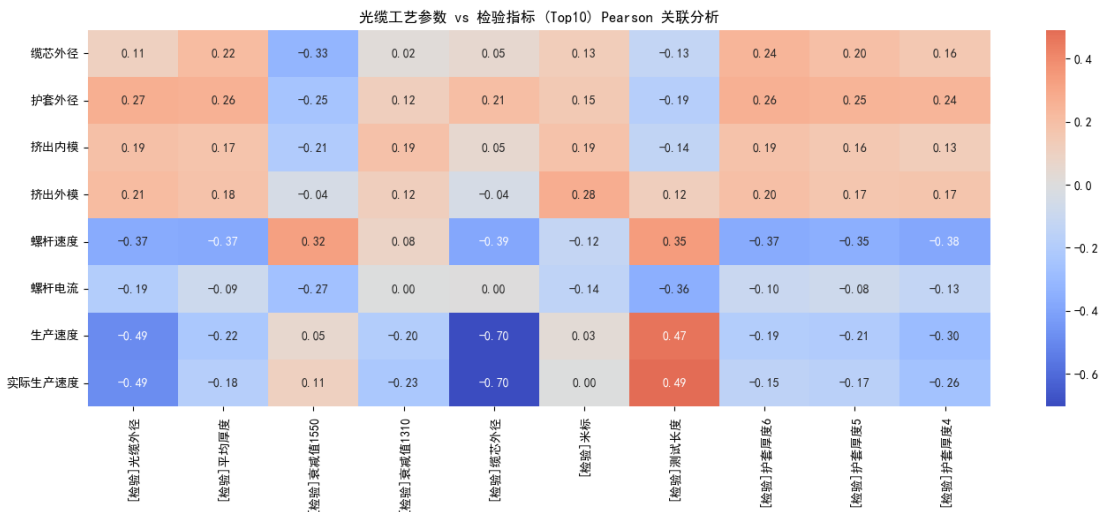
实验结果

运行代码，可获得 8x10 关联分析表：

8x10 关联分析表输出如下:

	[检验]光缆外径	[检验]平均厚度	[检验]衰减值1550	[检验]衰减值1310	[检验]缆芯外径	[检验]米标	[检验]测试长度	[检验]护套厚度6	[检验]护套厚度5	[检验]护套厚度4
缆芯外径	0.106	0.219	-0.327	0.018	0.050	0.130	-0.130	0.240	0.198	0.159
护套外径	0.270	0.260	-0.249	0.120	0.206	0.150	-0.185	0.263	0.251	0.240
挤出内模	0.193	0.175	-0.206	0.188	0.050	0.188	-0.136	0.187	0.158	0.128
挤出外模	0.215	0.180	-0.037	0.119	-0.044	0.275	0.122	0.199	0.166	0.173
螺杆速度	-0.372	-0.373	0.324	0.081	-0.386	-0.124	0.345	-0.370	-0.351	-0.383
螺杆电流	-0.191	-0.088	-0.267	0.001	0.004	-0.145	-0.355	-0.099	-0.081	-0.126
生产速度	-0.493	-0.224	0.053	-0.199	-0.704	0.030	0.467	-0.192	-0.209	-0.297
实际生产速度	-0.487	-0.184	0.108	-0.227	-0.703	0.000	0.491	-0.151	-0.166	-0.258

与关联分析热力图



四、实验总结与体会

(写写感想、建议等)

通过完成本次《实验 4 行为型设计模式应用及综合实战》，我完整地经历了一个工业数据分析系统从“面向过程的粗糙脚本”，一步步迭代为“面向对象的高内聚系统”，最终演进为“具备机器学习能力的流水线架构”的全过程。

在引入数据科学任务后，业务流程变得漫长且复杂（读取长表 → 透视为宽表 → 连接合并 → 处理缺失值 → 相关性分析 → 模型训练）。如果将这些代码平铺直叙，系统将变成无法维护的“面条代码”。

在实践中，我采用了模板方法模式构建了 `DataAnalysisPipeline`。它强制规定了数据的生命周期骨架，这使得我在后期调整“由于交集过少导致报错”的 `bug` 时，只需重写内部的局部方法，而不必担心破坏整体执行顺序。同时，结合策略模式，我将随机森林算法封装为 `IRegressionStrategy` 的实现。这遵循了开闭原则（OCP），如果未来客户提出想要改用深度学习或 `XGBoost` 来预测外径，我只需要新增一个策略类注入即可，主管道代码“零修改”。本次实验是对之前创建型、结构型模式的升华。它让我意识到，软件体系结构的设计不仅仅是为了代码“好看”，更是为了应对需求变更（如本次实验要求去除“操作手”逻辑、新增 `ML` 流水线）和脏数据冲击时，系统能够展现出的韧性与弹性。这种结合了软件工程理论与 `AI` 算法的综合训练，极大地提升了我的架构视野和实际工程能力。

五、成绩评定及评语

1.指导老师批阅意见:

2.成绩评定:

指导教师签字:毛斐巧

2026年 月 日